

《软件安全》实验报告

姓名：谢小珂

学号：2310422

班级：1069

实验名称：

格式化字符串漏洞

实验要求：

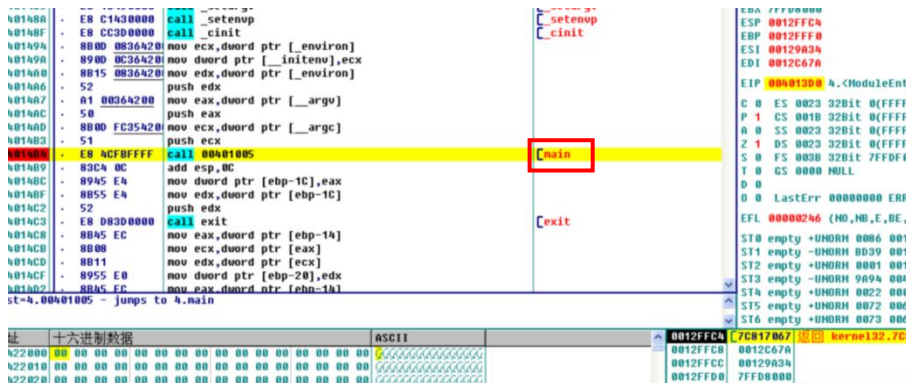
以第四章示例 4-7 代码，完成任意地址的数据获取，观察 Release 模式和 Debug 模式的差异，并进行总结。

实验过程：

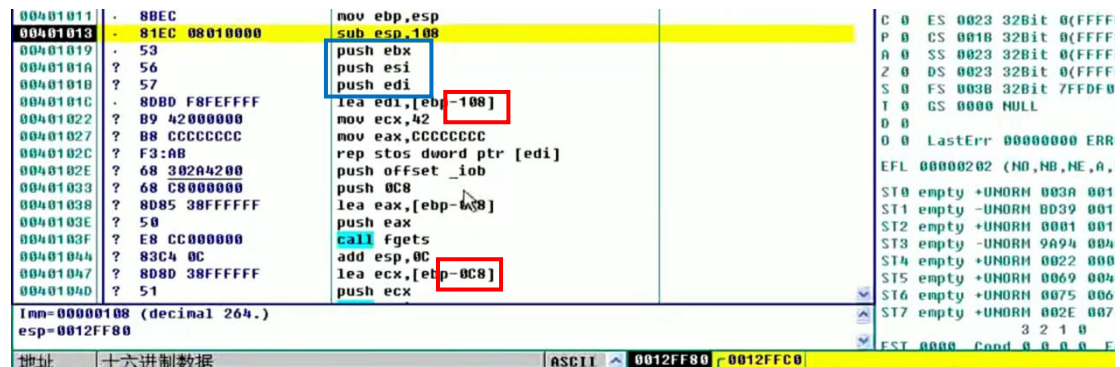
- 在 Debug 模式下开展测试，向目标程序输入字符串 “AAAA%x%x%x%x”

1. 先输入代码，接着在 DEBUG 模式下进行编译，从而生成 exe 文件。随后，使用 ollydbg 打开这个生成的 exe 文件，然后在其中向下查找，找到 main 函数所在的地址。

```
test.cpp
#include <stdio.h>
int main(int argc,char*argv[])
{
    char str[200];
    fgets (str,200,stdin);
    printf(str);
    return 0;
}
```



2. 使用 F7 键进行单步调试，程序进入函数内部执行。此时可以观察到，栈内存区域进行了较大范围的初始化操作。其中，sub esp,0x108 指令为局部变量分配了内存空间，而该空间的大小远超过了数组 str 实际所需的内存。经分析，数组 str 所需的内存仅为 0xc8。



3. 在程序执行过程中，依次执行 push ebx、push esi 以及 push edi 这三条入栈指令。这一系列操作的目的在于保存 ebx、esi 和 edi 这三个寄存器当前的状态。

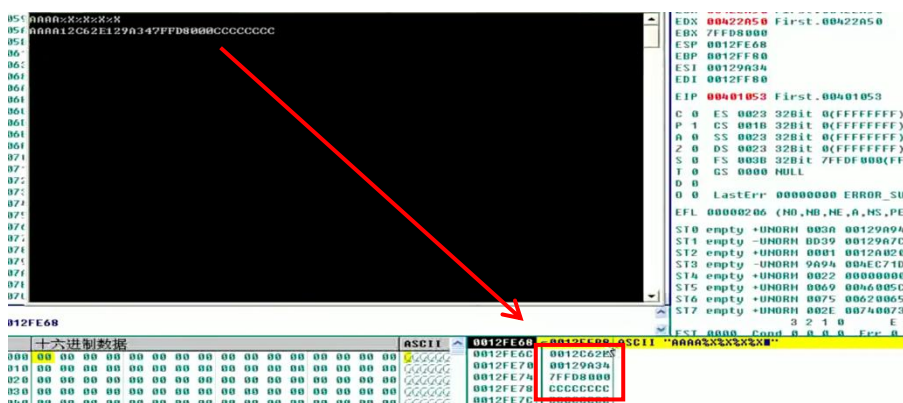
00401018	?	50	push esi
0040101B	?	57	push edi
0040101C	.	80BD F8FEFFFF	lea edi,[ebp-108]
00401022	?	B9 42000000	mov ecx,42
00401027	?	B8 CCCCCCCC	mov eax,CCCCCCCC
0040102C	?	F3:AB	rep stos dword ptr [edi]
0040102E	?	68 302A4200	push offset _iob
00401033	?	68 C8000000	push 0C8
00401038	?	8085 38FFFFFF	lea eax,[ebp-0C8]
0040103E	?	50	push eax
0040103F	?	E8 CC000000	call fgets
00401044	?	83C4 0C	add esp,0C
00401047	?	808D 38FFFFFF	lea ecx,[ebp-0C8]
0040104D	?	51	push ecx
0040104E	.	E8 3D000000	call printf
00401053	?	83C4 04	add esp,4

4. 程序执行 mov ecx 和 mov eax 指令，将栈区的初始值设定为 CCCCCCCC。随后，借助 rep stos 指令对大小为 0x108 的栈区进行循环赋值操作，以完成栈区的初始化工作。



5. 在调用 fgets 函数之前，程序执行了三次 push 指令，分别将三个参数压入栈中，当在终端输入字符串 AAAA%x%x%x%x 后，地址 0x0012FEB8 处存储的便是输入的这个字符串。

6. 程序持续执行，直至运行到 printf 函数处，此时字符串 AAAA%x%x%x%x 被压入栈中。当调用格式化输出函数 printf 时，鉴于输入字符串里存在格式化字符 %x，该函数会依据这些格式化字符来寻找对应的参数。具体而言，函数会先输出字符串字面内容 “AAAA”，随后从存储 “AAAA” 的地址 0x0012FEB8 开始，按顺序读取紧跟其后的四个内存地址处的数据，并将这些数据以十六进制的形式输出。



7. 在调试过程中，使用 F8 单步执行 printf 函数，进而得到最终的输出结果。在栈结构里，其首个位置存放着输入字符串的首地址，紧随其后的是主函数栈帧所保存的 edi、esi 以及 ebx 寄存器的值。

printf 函数会依据格式化字符串 %x%x%x%x 依次去寻找四个对应的参数。然而，由于我们并未显式提供这些参数，该函数便会在栈中自行查找。具体查找和输出情况如下：

函数找到的第一个数据是 0012C62E，按照十六进制格式输出，结果同样为 0012C62E。

接着找到的第二个数据是 00129A34，以十六进制形式输出后依旧是 00129A34。

随后找到的第三个数据是 7FFD8000，十六进制输出结果为 7FFD8000。

最后找到的第四个数据是 CCCCCCCC，十六进制输出为 CCCCCCCC。

从上述过程能够看出，程序在未获得明确参数的情况下于栈中自行搜索数据，可以判定发生了内存泄漏问题。

0012FEAC	CCCCCCCC
0012FEB0	CCCCCCCC
0012FEB4	CCCCCCCC
0012FEB8	41414141
0012FEBC	58255825
0012FEC0	58255825
0012FEC4	CCCC000A
0012FEC8	CCCCCCCC

8. 若在输入字符串的末尾追加“%S”，便能够将位于“AAAA”这一任意构造地址处的数据打印输出。我们找到“AAAA”地址，可观察到在已定义的 200 字节区域前后，均填充有“cccccccc”。这一现象是由于在 Debug 模式下，系统所分配的内存空间相较于局部变量所定义的空间显著更大，从而导致了这种超出预期的内存填充情况。

• 在 Release 模式下开展测试，向目标程序输入字符串“AAAA%x%x%x%x”

1. 进入 release 模式，观察差异，找到 arg 变量，从而找到主程序的入口。

004011F5	. A3 C4684000	mov dword ptr [4068C4],eax	
004011FA	. E8 13110000	call 00402312	[Test.00402312
004011FF	. E8 55100000	call 00402259	[Test.00402259
00401204	. E8 8D0C0000	call 00401EC6	[Test.00401EC6
00401209	. A1 00694000	mov eax,dword ptr [406900]	
0040120E	. A3 04694000	mov dword ptr [406904],eax	
00401213	. 50	push eax	
00401214	. FF35 F868400	push dword ptr [4068F8]	[Arg3 => [406900] = 0
0040121A	. FF35 F468400	push dword ptr [4068F4]	[Arg2 = 0
00401220	. E8 DBFDFFFF	call 00401000	[Arg1 = 0
00401225	. 83C4 0C	add esp,0C	[Test.00401000
00401228	. 8945 E4	mov dword ptr [ebp-1C],eax	
0040122B	. 50	push eax	
0040122C	. E8 C20C0000	call 00401EF3	
00401231	. 8B45 EC	mov eax,dword ptr [ebp-14]	

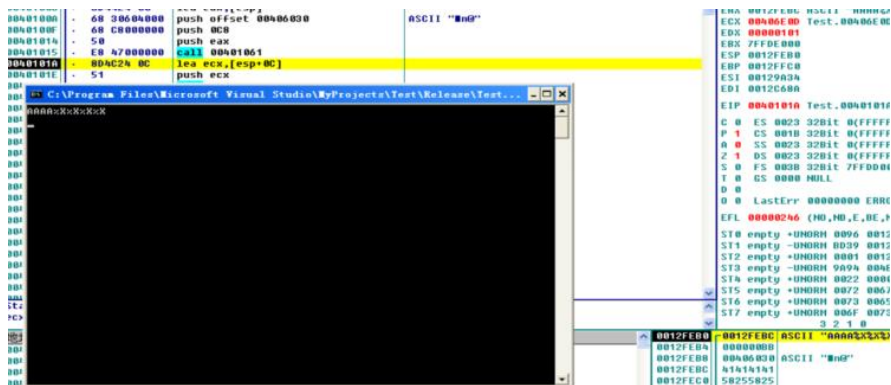
2. 在调试过程中，当执行到 call 00401000 语句时，按下 F7 键，程序会直接跳转至主函数的地址，期间未发生额外的 jump 操作，同时返回地址被压入栈中。

对主函数的栈帧切换情况进行观察，可发现其与常规情况存在差异。通常，栈帧切换会将基址指针 ebp 压入栈，但此主函数并未执行该操作。接着执行 sub esp, 0xc8 指令，这意味着栈顶向上抬高的幅度相较于 DEBUG 模式下的 sub esp, 0x108 有所减小，仅抬高了 0xC8（即 20 字节），其目的是为局部变量分配内存空间。

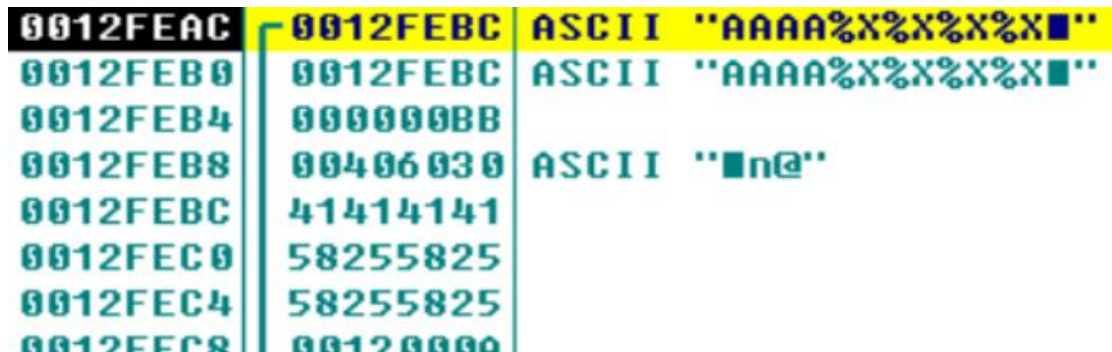
0040100F	. 68 C8000000	push 0C8	
00401014	. 50	push eax	
00401015	. E8 47000000	call 00401061	← 输入字符串
0040101A	. 8D4C24 0C	lea ecx,[esp+0C]	
0040101E	. 51	push ecx	
0040101F	. E8 0C000000	call 00401030	
00401024	. 33C0	xor eax,eax	
00401026	. 81C4 D800000	add esp,0D8	
0040102C	. C3	ret	
0040102D	. 90	nop	
0040102E	. 90	nop	
0040102F	. 90	nop	

此外，DEBUG 模式下常见的诸多 push 寄存器值的操作在此处并未出现，这使得代

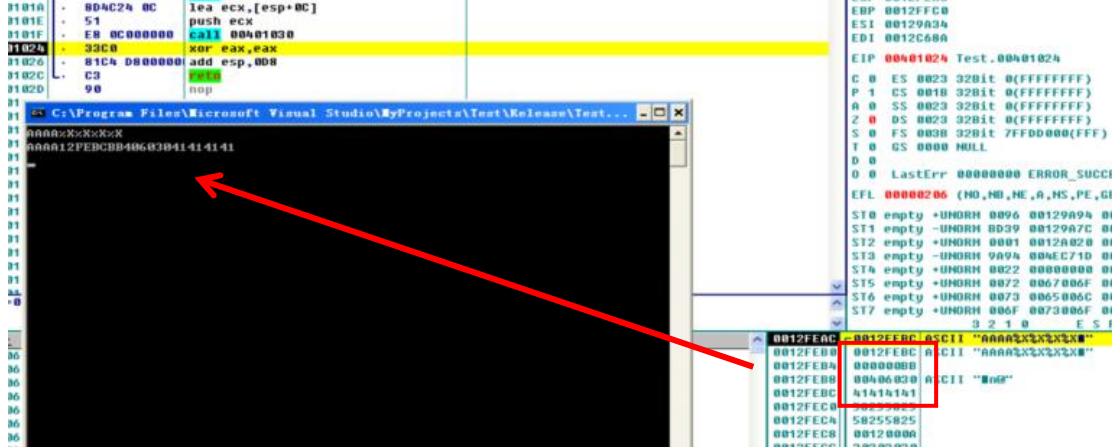
码结构更为简洁，有助于提升程序的运行效率。当输入 AAAA%x%x%x 并结束操作后，在 DEBUG 模式下会执行 add eip 操作，然而在 Release 模式下则不会出现这一操作。



3.在调用 printf 函数之前，经过计算得出字符串的地址为 0012FEB0。通过对该地址进行查看，可以发现其中存储着字符串对应的 ASCII 码值。



4. 输出结果与 DEBUG 模式存在差异，最终呈现为 AAAA12FEB0BB40603041414141。与此类似，若输入的字符串设定为 AAAA%x%x%x%s，那么输出结果的末尾部分将会是地址 41414141 所指向内存区域中的特定数据。这种现象反映出不同输入格式以及不同调试模式对程序输出结果的影响，体现了内存数据在不同条件下的呈现方式。



• Debug 模式和 Release 模式的差异

1. 内存分配:

Debug 模式：分配更多内存如 sub esp, 0x108，填充 CCCCCC，便于检测错误。

Release 模式：精确分配如 sub esp, 0xC8，无冗余填充，效率更高。

2. 函数调用:

Debug 模式: 保留完整栈帧如保存 ebp、ebx 等寄存器, 便于调试。

Release 模式: 优化栈帧 (可能省略 ebp), 减少指令, 提升性能。

3. 格式化字符串漏洞表现:

Debug 模式: 输出固定如寄存器值、CCCCCCCC, 便于分析。

Release 模式: 输出可能混乱如 AAAA12FEB40603041414141, 因优化导致栈布局变化。

Debug 模式适合安全测试, Release 模式更接近真实环境, 两者差异影响漏洞利用的稳定性。

心得体会:

通过本次关于格式化字符串漏洞的实验, 我理解了该漏洞的原理及其在不同编译模式下的表现差异。在 Debug 模式下, 由于编译器保留了完整的调试信息并进行了额外的内存初始化, 使得漏洞的利用和分析更加直观。例如, 填充的 CCCCCCCC 和固定的栈布局让我能够清晰地观察到内存泄漏的过程。而在 Release 模式下, 优化后的代码虽然提升了运行效率, 但也使得栈布局变得难以预测, 增加了漏洞利用的复杂性。

这次实验让我认识到, 在实际的软件安全测试中, 必须考虑不同编译环境对漏洞的影响。Debug 模式适合初步分析和漏洞挖掘, 而 Release 模式更接近真实场景, 需要更细致的调试技巧。此外, 格式化字符串漏洞的危害性也让我意识到, 开发中必须严格检查用户输入, 避免使用不安全的函数如 printf 直接输出用户输入, 以增强程序的安全性。

总的来说, 本次实验不仅加深了我对漏洞原理的理解, 还提升了我的实践能力和安全意识, 为今后的安全研究打下了坚实的基础。